Name:- Aaryan N. Raorane Roll No:- 5004 Branch:- AIDS
Batch:- A1 SAP ID:- 60018250070 Div:- 5

Experiment No. 8 06/10

Title:- Built-in functions and User defined functions.

AIM:- To understand and apply built in and user defined functions in Python programming and explore their usefulness in solving problems.

Theory:-

1- What are the advantages of using functions in Python Programs?

=> Functions allow a block of code to be written once and used many times, which removes repeated code and keeps the program short. A large problem can be divided into smaller functions, so each part is easier to read, test and debug. Any change made only inside a function takes effect wherever it is called, which makes maintenance simple. Function take inputs through parameters and sent results back with return, so the same logic works for different data. Python provides built in functions such as len, max and sum for common tasks, and the programmer writes user defined functions with the def keyword for tasks specific to the program.

Sundaram FOR EDUCATIONAL USE

code :-

```
numbers = [4,9,2,7]

print(len(numbers), max(numbers), sum(numbers))

def area(r):
    return 3.14159 * r * r

print(area(2))
print(area(5))
```

output :-

```
4  9  22
12.56636
78.53975
```

Q2 Differentiate between positional arguements, keyword arguements, and default arguements with examples.

⇒

Point	Positional Arguements	Keyword Arguements	Default Arguements
Meaning	Values are matched to Parameters by their Position	Values are passed along with the parameter name	Parameter has a preset value that is used when no value is passed.

Sundaram
FOR EDUCATIONAL USE

Order	Values are matched to parameters by their order must be followed	Orders does not matter	Must come after the parameters without defaults
Syntax	f(10,20)	f(a=10, b=20)	def f(a,b=20)
Use	short and simple calls	clear calls when there are many parameters	Makes some arguments optional

code:

```
def student(name,age, course='AI and DS')
    print(name, age, course)

student("Riya", 19)                          # positional
student(age=19, name="Riya")                 # Keyword
student("Riya", 19, "cyber security")        # replaces the default.
```

output:

```
Riya  19  AI and DS
Riya  19  AI and DS
Riya  19  Cyber Security.
```

Sundaram
FOR EDUCATIONAL USE

Q 3 What is recursion? How does it differ from iteration?

→ Recursion is a technique in which a function calls itself to solve a smaller version of the same problem until a base case is reached. The base case stops the calls, and the result are then combined as the calls return. Iteration repeats a block of statements using a for or while loop until its condition becomes false.

Point	Recursion	Iteration
working	Function calls itself	Loop repeats the same statements.
Termination	Stops when the base case is reached	stops when the loop condition becomes false
Memory	Each call uses a new stack frame, so more memory is needed.	uses a fixed amount of memory.
speed	Slower because of repeated function calls	faster in general.
Code	shorter and closer to the mathematical definition	usually longer.

Sundaram®
FOR EDUCATIONAL USE

Code :-

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n-1)

def factloop(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result

print(factorial(5))
print(factloop(5))
```

output :-

```
120
120
```

Q4 Compare lambda functions with normal user-defined functions.

⇒

Point	Lambda function	Normal function
Keyword	lambda	def

Sundaram®
FOR EDUCATIONAL USE

Name	Anonymous, but can be assigned to a variable	Has a name given in the definition.
Body	Only a single Expression	Many statements are allowed
Return	Value of the expression is returned automatically	return statement is used
Use	Short operations needed once	Reusable and longer logic

Code:

```
square = lambda x: x * x
add = lambda a, b: a + b

def squaredef(x):
    return x * x

print (square(6), add(3, 4))
print ( squaredef(6))
print ((lambda x: x + 10)(5))
```

Output:- 36 7
36
15

Sundaram FOR EDUCATIONAL USE

Q5 Discuss the difference between local and global scope in relation to Python functions.

→ A variable created inside a function has local scope, so it exists only while the function runs and cannot be used outside it. A variable created outside all functions has global scope and can be read anywhere in the program. When a function needs to change a global variable it must declare it with the global keyword, otherwise Python creates a new local variable with the same name.

Point	Local scope	Global scope
Defined	Inside a function	Outside all function.
Lifetime	Till the function finishes	Till the program ends.
Access	Only inside that function	Everywhere in the program
Change inside a function	Directly possible	Needs the global keyword.

Sundaram
FOR EDUCATIONAL USE

Code:

```
x = 10
def show():
    x = 5
    print("inside", x)

def change():
    global x
    x = 20

show()
print("outside", x)
change()
print("after change", x)
```

Output:

```
inside 5
outside 10
after change 20
```

Conclusion:

Built in and user defined functions were studied and used to solve small problems in Python. Positional, Keyword and default arguments, recursion and lambda functions were implemented and their differences were observed. Local and global scope decide where a variable can be used, and the global keyword lets a function change a global variable.

Sundaram
FOR EDUCATIONAL USE